

Prescripto

Comprehensive Smart Healthcare Platform: Architecture & Implementation Roadmap

Project Architect: Sarthak Kandpal

Document Type: System Design & Development Masterplan

Executive Summary: Prescripto is a next-generation healthcare application designed to optimize medical routing, enhance patient triage through Artificial Intelligence, and centralize medical record management. This document outlines the technological foundation and step-by-step development phases required to bring the platform from concept to production.

1. Core Platform Features

The platform is designed to handle both emergency logistics and day-to-day healthcare management through the following modules:

- **One-Button Emergency Protocol:** Instantly captures user coordinates, evaluates medical urgency, cross-references hospital budget/specialty, and initiates ambulance dispatch or third-party rideshare (Ola/Uber) via deep links.
- **AI Triage & Chatbot:** A robust AI assistant leveraging Retrieval-Augmented Generation (RAG) to provide initial medical guidance and recommend targeted medical tests based on user symptoms.
- **Intelligent Appointment System:** An availability-based booking system featuring automated SMS notifications for schedule delays and an advanced hospital recommendation engine based on problem type, distance, and budget.
- **Unified Medical Profiles:** Secure, cloud-based storage for patient medical reports, made accessible to consulting doctors for seamless review.
- **Real-time Logistics:** Live tracking of ambulance placements and active monitoring of hospital bed occupancy to ensure patients are routed to facilities with immediate availability.

2. Recommended Technology Stack

To ensure scalability, real-time processing, and deep AI integration, a decoupled architecture is required. The following stack separates the fast, responsive frontend from the heavy machine-learning processes of the backend.

Domain	Technology	Justification
Frontend	Next.js (React) + TailwindCSS	Provides exceptional performance through Server-Side Rendering (SSR) and dynamic routing for complex, data-heavy dashboards.
Backend API	FastAPI (Python)	Python is mandatory for native ML integration. FastAPI provides high-performance, asynchronous endpoints critical for real-time routing.
Database	Neon DB (Serverless PostgreSQL)	Highly scalable. Crucially supports the pgvector extension, which is required to store document embeddings for the RAG architecture.
Mapping & Routing	Ola Maps API	Utilized for Distance Matrix calculations (ETA, proximity algorithms) and Directions API for real-time ambulance tracking.
AI / NLP Framework	LlamaIndex + LLM	Streamlines document ingestion and query handling for the triage chatbot, interfacing directly with Neon DB vectors.
Communication	Twilio API	Ensures reliable, automated SMS delivery for appointment delays or emergency confirmations.
File Storage	AWS S3 / Supabase Storage	Highly secure, scalable blob storage for sensitive patient PDF medical reports.

3. Phased Development Roadmap

The project should be executed in discrete phases, establishing the data foundation first before layering on intelligent routing and AI interactions.

Phase 1: Foundation & Data Engineering

- **Synthetic Dataset Creation:** Curate an Excel dataset representing realistic hospital entities, including exact geographical coordinates, bed capacities, financial tiers, and specialized departments.
- **Database Initialization:** Deploy Neon DB. Architect relational schemas for Users, Hospitals, Appointments, and Reports. Initialize a dedicated vector table for AI embeddings.
- **Data Pipeline:** Develop Python ingestion scripts (using pandas) to parse the Excel synthetic data and populate the PostgreSQL tables automatically.

Phase 2: User Access & Record Management

- **Authentication:** Implement secure user and doctor login workflows using NextAuth.js.
- **Cloud Report Storage:** Develop the patient dashboard. Enable file uploads directly to AWS S3/Supabase, storing secure retrieval URLs back into the Neon database.
- **Provider Portal:** Build the physician-facing UI allowing secure, read-only access to patient histories and linked reports prior to appointments.

Phase 3: The "One-Button" Emergency Engine

- **Geolocation Trigger:** Implement Next.js frontend logic to capture highly accurate, real-time browser/device GPS coordinates.
- **Prediction Algorithm:** Build the FastAPI endpoint that filters the database for hospitals matching the user's budget and specific medical condition.
- **Logistics Integration:** Feed the filtered hospital list into the Ola Maps Distance Matrix API to calculate the absolute shortest ETA.
- **Dispatch System:** Build the dual-option UI allowing users to request an in-app simulated ambulance or trigger mobile deep links for immediate Uber/Ola routing to the selected hospital.

Phase 4: RAG Chatbot & Automated Triage

- **Knowledge Base Assembly:** Gather standard medical triage guidelines and symptom checkers. Process text into vector embeddings and store in Neon's `pgvector` table.
- **Query Pipeline:** Utilize LlamaIndex to retrieve relevant medical context based on the user's conversational input.
- **Test Recommendation Model:** Prompt the LLM to analyze the retrieved context against the user's input to suggest probable causes and recommend specific medical tests, returning structured JSON to the frontend.

Phase 5: Standard Appointments & Notifications

- **Slot Management:** Develop the Next.js booking calendar. Implement transaction locking in PostgreSQL to prevent concurrent double-booking of doctors.
- **Elective Recommendations:** Adapt the Phase 3 algorithm to recommend hospitals for non-emergencies, heavily weighting budget alignment and specialized care over sheer distance.
- **Twilio Integration:** Create asynchronous background tasks in FastAPI that listen for schedule modifications and trigger SMS alerts to the patient's registered mobile number.

Phase 6: Real-Time Fleet & Bed Tracking

- **Occupancy Simulation:** Write a scheduled Python background script that dynamically alters hospital bed counts to mimic real-world fluctuations.
- **Live Telemetry:** Implement WebSockets in FastAPI to push simulated, continuous GPS coordinate updates to the Next.js frontend, rendering smooth ambulance movement on an Ola Map component.

4. Next Steps

The immediate priority is the establishment of the Neon PostgreSQL database and the ingestion of the synthetic hospital data. Once the core data layer is accessible, backend routing logic and frontend map integration can proceed simultaneously.